PREDICTIVE AND ADAPTIVE VIRTUAL MEMORY MANAGEMENT

with Intelligent Thrashing Prevention

OPERATING SYSTEMS LAB PROJECT REPORT

Submitted by
Aditya Kiran Upadhye
Aman Singh Sachan
Aditya Jain

A reproducible simulation of address translation,
page replacement, thrashing detection, and adaptive control.

1. Executive Summary

This project implements an educational virtual-memory simulator. It models address translation through a translation lookaside buffer (TLB) and a two-level page table, simulates page faults and replacement, estimates working-set pressure, detects thrashing using rule-based thresholds, and demonstrates adaptive frame redistribution and temporary process suspension.

The committed experiment has 1000 rows across five synthetic workload patterns, ten seeds, four replacement policies, four adaptive-control modes, and matched TLB checks. Under LRU, combined adaptation reduced pooled mean faults from 1,193.1 to 963.6 (19.2% by ratio of means). The mean paired reduction is 22.3%. These are simulator results, not measurements from a real operating system.

Objectives

- Demonstrate a full virtual-address translation path with TLB and hierarchical page tables.
- Compare FIFO, LRU, Clock, and Optimal page replacement on repeatable reference streams.
- Estimate working-set pressure and detect simulated thrashing.
- Evaluate frame redistribution and process suspension independently and together.
- Publish run-level CSV output, summaries, figures, tests, and a reproducible report.

Scope

The simulator does not inspect or modify host OS memory, kernel page tables, live CPU usage, or real running processes. CPU utilization, pressure, and process suspension shown in the dashboard are simulated proxies.

2. System Architecture

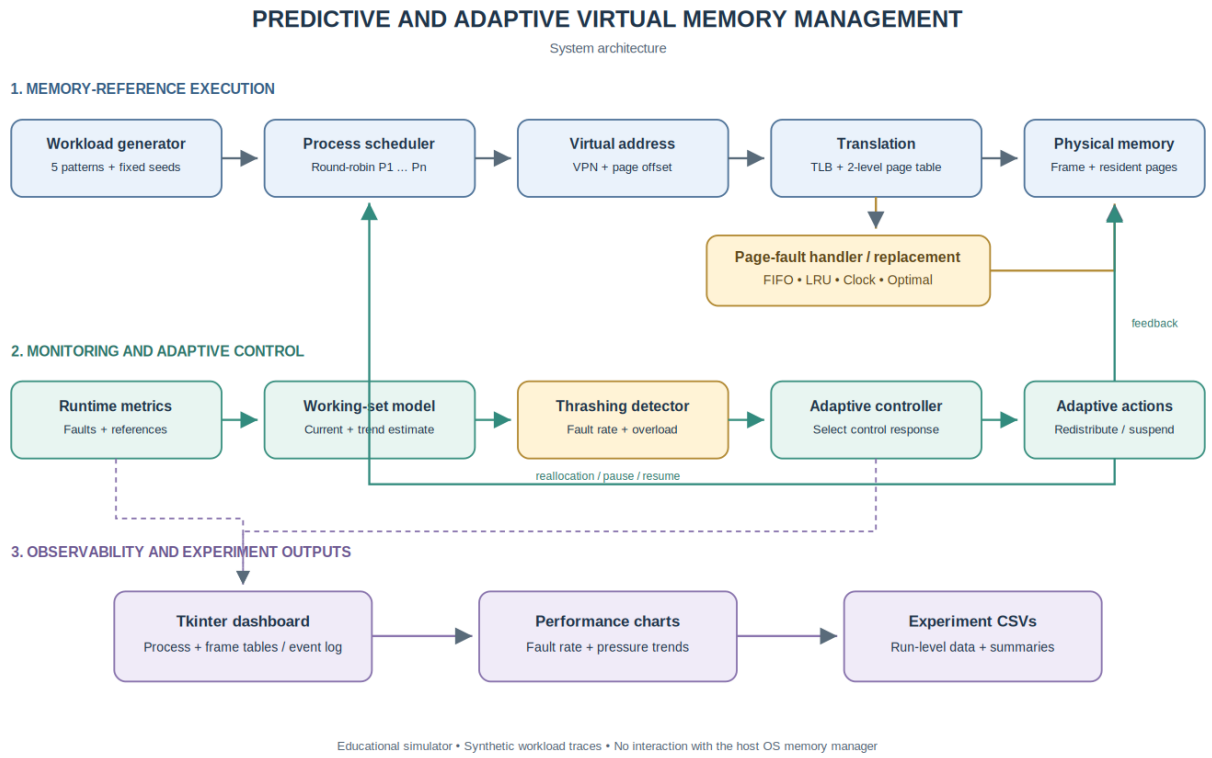


Figure 1. Workload execution, feedback control, and experiment outputs.

A workload generator supplies synthetic references to a round-robin scheduler. Address translation checks the TLB before walking the two-level page table. A non-resident page invokes a fault handler and the selected replacement algorithm. Runtime metrics feed a working-set estimator and thrashing detector; the adaptive controller can change frame ownership and temporarily suspend a process.

Module	Responsibility
vm_simulator/core.py	TLB, page tables, physical frames, replacement, workloads, monitoring, and adaptive controls.
vm_simulator/gui.py	Tkinter dashboard, charts, process/frame tables, event log, address inspector, CSV export.
benchmark.py	Multi-seed experiment, ablations, confidence-interval summaries, paired effect analysis, TLB comparisons.
docs/generate_figures.py	Rebuild figures from the experiment CSVs and render the architecture diagram.

3. Virtual-Memory Model

Address format and translation

The modeled page size is 256 bytes; the offset is the low eight bits. The 64 virtual pages use two 3-bit page-table indices, with eight entries at each level, giving a virtual address range of 0 through 16,383.

Stage	Action	Metric
TLB lookup	Lookup cached translation by (process, VPN).	TLB hits and misses.
Page-table walk	Check first- and second-level entries.	Page-table accesses.
Page fault	Select a frame allocated to process; evict if occupied; map VPN.	Faults and replacements.
Physical address	Combine frame number and page offset.	Address shown in event log.

Replacement policies

Policy	Summary
FIFO	Evict the page loaded earliest.
LRU	Evict the least recently used resident page.
Clock	Use reference bits to implement a second-chance scan.
Optimal*	Evict the resident page whose next use is farthest away.

*Optimal uses knowledge of future generated references and is only a theoretical oracle comparison, not an implementable online policy.

4. Thrashing Detection and Adaptive Control

A recent-reference window estimates each process's working set. A trend heuristic estimates possible near-term growth in demand. Pressure compares active working-set demand to frame capacity. Thrashing detection combines a rolling page-fault ratio and an overload condition. These are hand-set rules, not a trained machine-learning model.

Mode	Redistribution	Suspension	Interpretation
none	Off	Off	Baseline.
redistribution_only	On	Off	Isolates effects of frame allocation changes.
suspension_only	Off	On	Suspended process retains its allocation, allowing suspension effect to be isolated.
combined	On	On	Suspension and reallocation can coordinate.

Suspension-only page-fault counts therefore remain equal to baseline in this implementation. In combined mode, frame redistribution can transfer the suspended process's frames to active processes. The outcome is workload-dependent.

Dashboard

The dashboard shows process state/faults, frame ownership/resident pages, recent fault rate, TLB hit rate, pressure estimate, simulated CPU-use proxy, and an event log. The address inspector is read-only and does not advance simulation time or increment counters.

5. Dashboard Layout Illustrations

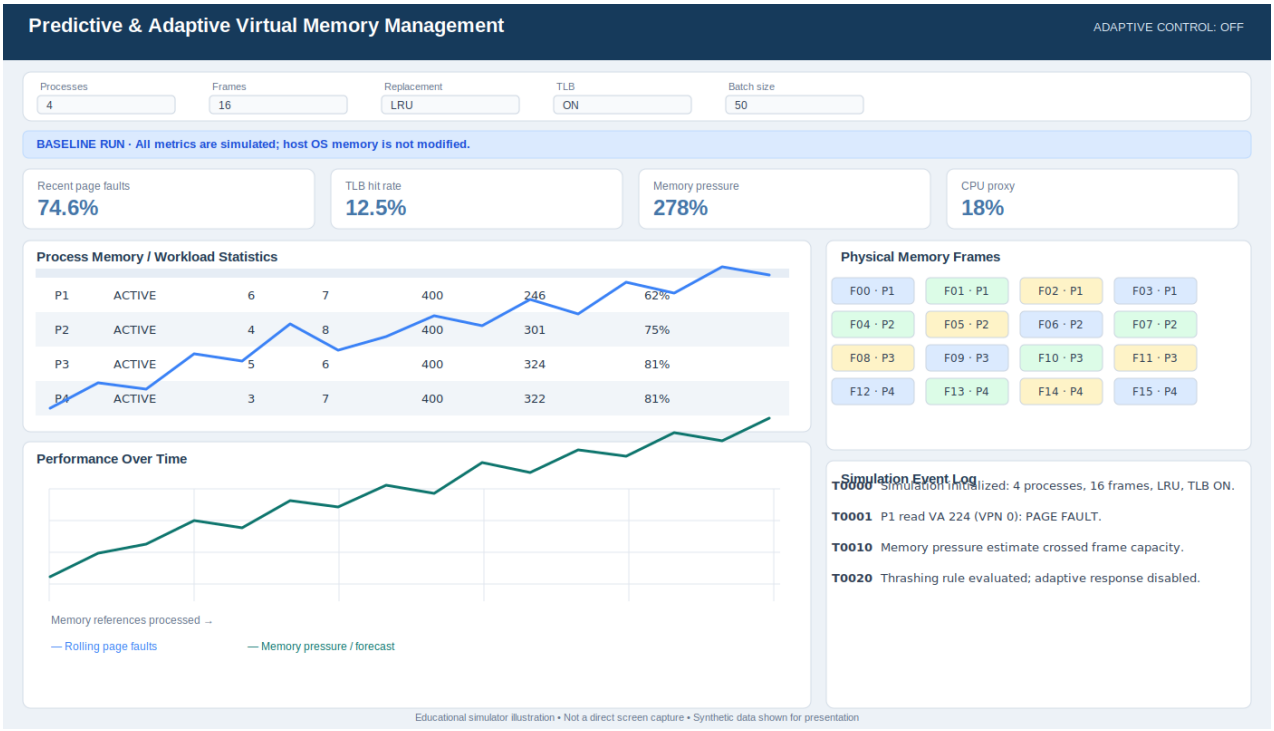


Figure 2. Redrawn baseline dashboard layout; illustrative, not a direct capture.

This dashboard illustration reflects the baseline layout and labels; displayed metrics are illustrative values, not a recorded experiment.

5. Dashboard Layout Illustrations (continued)

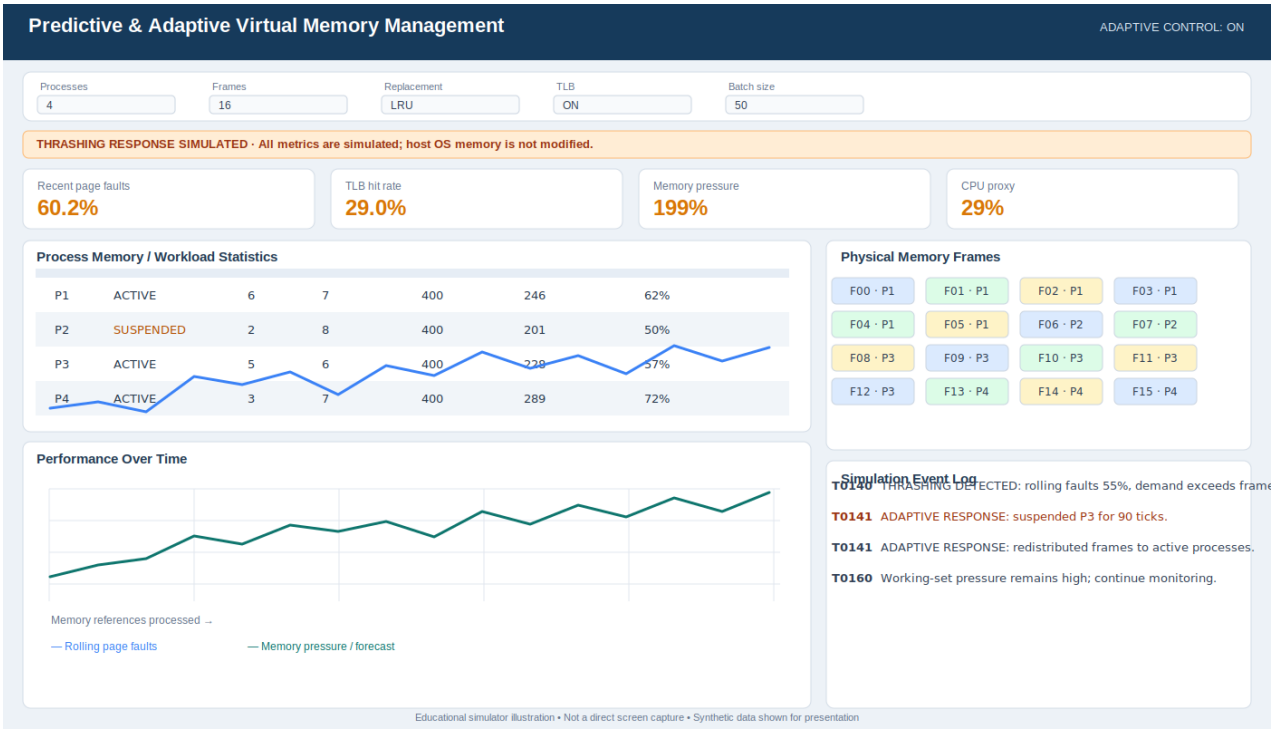


Figure 3. Redrawn adaptive dashboard layout; illustrative, not a direct capture.

This dashboard illustration shows the adaptive-control presentation; values are illustrative and no real host process is suspended.

6. Experimental Methodology

Parameter	Setting
Processes	4
Physical frames	16
References per process	400
References per completed main run	1,600
Seeds	42–51 (10 seeds)
Workload patterns	5
Replacement policies	FIFO, LRU, Clock, Optimal
Adaptive modes	4
Main TLB-on runs	800
Matched TLB-off runs	200
Total raw records	1000

Workload	Trace structure
phase locality	Mostly accesses a small region, then shifts by phase.
uniform random	Random accesses across 64 pages.
sequential scan	Sequentially scans all 64 pages.
bursty hot set	Small hot-set bursts interspersed with broad random phases.
alternating hot sets	Alternates between two disjoint eight-page regions.

The main design varies pattern, seed, replacement policy and adaptive mode. The TLB validation repeats no-adaptation cases with TLB disabled. Summary intervals use a normal approximation and should be treated as descriptive.

7. Aggregate Ablation Results

Policy	None	Redistribution	Suspension	Combined	Reduction*
FIFO	1199.8	1206.7	1061.2	965.8	19.5%
LRU	1193.1	1201.9	1050.5	963.6	19.2%
Clock	1197.5	1205.5	1051.6	964.2	19.5%
Optimal	953.4	974.2	869.5	843.6	11.5%

Table 1. Mean page faults pooled over seeds and patterns; all runs complete 1,600 references.

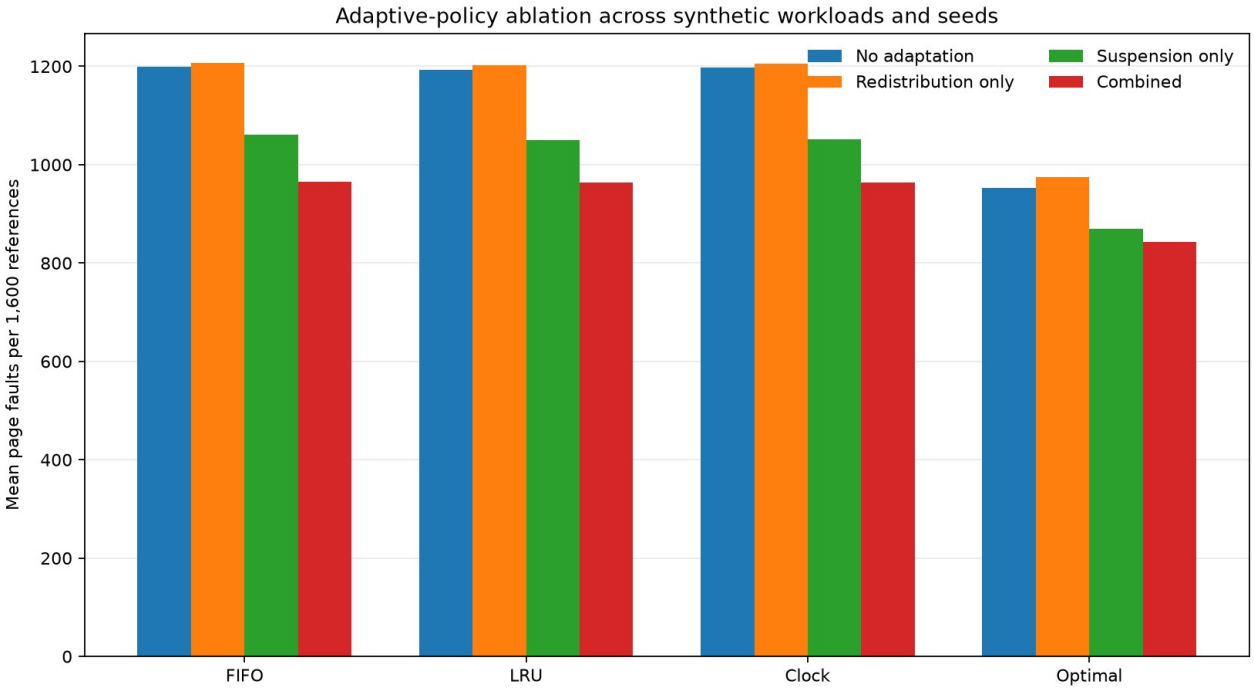


Figure 4. Adaptive-policy ablation.

Redistribution alone slightly increased mean faults, partly because changing frame ownership can invalidate useful resident pages. Suspension alone kept page-fault totals unchanged because it did not transfer frames. The combined mode reduced pooled faults for all four replacement algorithms in this synthetic experiment.

8. Workload-Specific Findings and TLB

Workload	Baseline LRU	Combined LRU	Reduction
phase locality	1026.1	676.3	34.1%
uniform random	1498.3	1272.7	15.1%
sequential scan	1600.0	1600.0	0.0%
bursty hotset	966.5	680.0	29.6%
alternating hotsets	874.5	589.2	32.6%

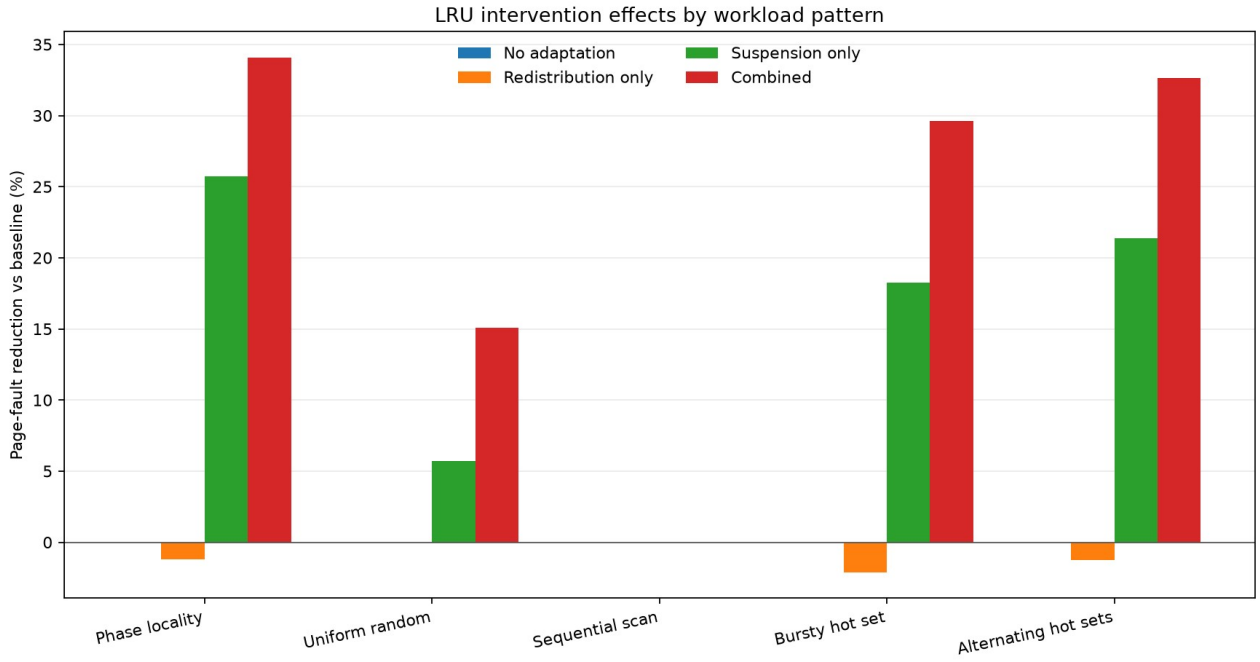


Figure 5. LRU intervention effect by workload; negative values indicate more faults.

TLB measure	Result
Mean table accesses, TLB on	2,450.2
Mean table accesses, TLB off	2,856.6
Reduction in table accesses	14.2%
Page-fault mismatches in matched pairs	0 / 200

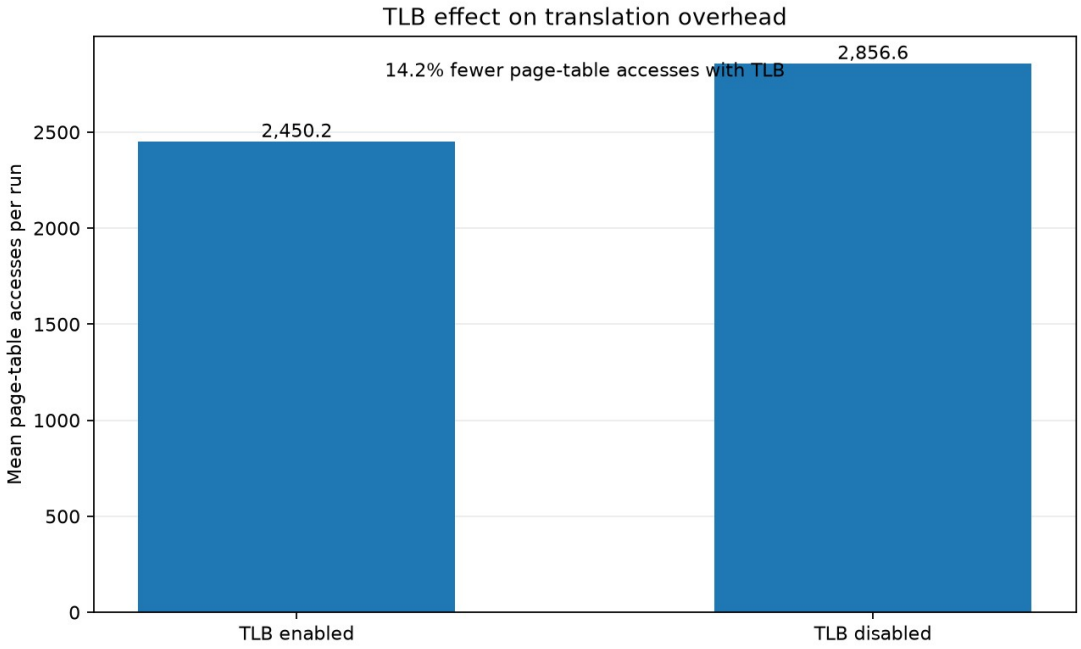


Figure 6. Translation-table access overhead, TLB enabled vs disabled.

The TLB caches translations rather than page contents. Accordingly, it reduces page-table accesses but should not change whether a page is resident. The paired dataset is checked for page-fault mismatches.

9. Reproducibility and Testing

- Install dependencies from requirements.txt in a Python virtual environment.
- Run python -m unittest discover -s tests -v to execute automated tests.
- Run python benchmark.py to regenerate the raw data, grouped summary, and paired analysis.
- Run python docs/generate_figures.py to regenerate chart images and the architecture PNG.
- Run python docs/build_report.py to rebuild the Word report; the GitHub Actions workflow exports it to PDF.

Validation coverage

The test suite covers two-level mapping and unmapping, TLB behavior, all replacement algorithms, allocation invariants, address-inspector validation, all workload generators, and independent adaptive switches. Run-level and grouped CSVs expose intermediate data for review.

10. Limitations and Conclusion

- Workloads are synthetic, not collected from real applications or operating systems.
- Pressure and CPU utilization are simulated proxies, not host telemetry.
- Prediction and thrashing thresholds are rule-based heuristics.
- Suspension-only retains frame ownership and therefore does not model OS reclamation.
- Redistribution can increase faults, and sequential scans may not benefit from the selected policy.
- Optimal uses future references and is only an oracle baseline.
- Further work should test threshold sensitivity, more seeds, real traces, reclamation-aware suspension, and additional outcomes such as completion time.

Conclusion

The project combines memory translation, replacement algorithms, working-set reasoning, and a visible simulation of feedback control. Its main strength is reproducibility: the same workloads can be used for paired comparisons, individual adaptive mechanisms can be isolated, and results are retained as CSV. The experiment shows that coordinated controls can reduce faults for some synthetic workload patterns but not all; the sequential-scan result illustrates why adaptive policies require workload-aware evaluation.

Reference basis

Policy terminology follows standard operating-systems concepts for paging, TLBs, FIFO/LRU/Clock replacement, working sets, and thrashing. Quantitative claims in this report are computed from the accompanying CSV files.